

Assay: the full report

Badger BuildFest 2026 · UW–Madison · Applied AI & Automation track

Challenges: Databricks Real-World Workflows (Xorbix) · The Art of the Break

Status as of **Saturday 26 September 2026, 20:55 CDT** (submissions close Sunday 11:00).

	Link
Code	https://github.com/Ananda-001/Badger-Build-Fest (branch <code>main</code>)
Live app on Databricks	https://assay-7474649367590010.aws.databricksapps.com (Databricks login required; stops 24 h after each deploy)
Manager UI mock (the design we are moving to)	https://claude.ai/artifact/MURyj2T5F6QH6ZC4JqfHad
Verdicts in one page	<code>results/stage-2-3-runs/VERDICTS.md</code>

1. Summary in one minute

Assay is a manager for AI agents. It sits between the AI agents a company already uses and the people who review their work, and it decides, **with evidence**, how much each agent may do on its own.

It does four things, all of them working today on real data and real models running on Databricks:

- 1. Act alone or ask?** An agent may act without a human only where its checked track record proves it.
Result: a small model said it was 95%+ sure on 19 duplicate tickets and was right on 4. Assay refused it.
- 2. Which model answers?** Assay switches models live, per request: the cheaper model only where it is certified, and a backup when a model is overloaded or returns junk. *Result:* 36 live requests on Databricks, including one real switch (Llama 70B busy → Qwen 80B), whose answer was held for review because Qwen isn't certified yet.
- 3. Did a correction really help?** Before a new rule or prompt is switched on, Assay tests it on fresh cases.
Result: a plausible rule ("version upgrades are duplicates") broke 16 decisions and fixed none. Blocked.
- 4. Reuse past reviewer decisions.** When reviewers have decided a kind of case consistently enough to prove it, Assay applies their answer without asking. *Result:* one precedent (21 of 21 rejections) is about 8 reviews away from being proven at our 90% bar.

The demo agent tidies up the **public Apache Jira** (Spark, Kafka, Flink, Hive): it spots tickets that are the same problem, or that belong under a bigger ticket. **37,853 real tickets.** Almost half of the tickets closed as duplicates were never linked to their original: knowledge the agent can recover.

What we refuse to claim: no model has earned the right to act alone; the cost question is not certified; the labels behind two of the three relation types are too inconsistent to rely on. Section 13 lists every limit.

2. The product, as we would explain it to a business owner

What is it? An AI agent manager: a middle layer between your AI agents and your human reviewers.

What problem does it solve? Companies either trust an AI agent completely (and clean up its mistakes) or check everything it does (and lose the time the agent was meant to save). The usual compromise, "send low-confidence cases to a human", fails because **the AI's confidence is not reliable**. We measured it: 95% claimed, 21% real.

How does it decide? From checked results, never from the model's own confidence:

- a kind of decision goes on autopilot only when enough past decisions of that kind were checked and proven right (a statistical lower bound clears a target, 90% by default);
- everything else goes to a person, in plain words, with Yes / No / Not sure buttons;
- every answer a person gives is stored as evidence and moves that kind of decision closer to autopilot.

Does it train itself? It learns, but it never retrains the AI model (models can't change their own weights, so a model alone repeats the same mistakes). Assay learns *around* the model: which decisions no longer need a person, which corrections actually help, which model is safe for which task.

Does it choose the model? Yes, live, per request. A cheaper model only gets work Assay has certified it for; an overloaded or broken model is replaced on the spot; every choice is logged with a reason.

What does the manager see? One dashboard (being built from the mock): what the agent did today, what needs your OK, how close each task is to autopilot, what was held back for safety, and what it saved. A guided tour explains the page on first visit.

The end-product vision (not built for the demo): agents call Assay's API instead of the model; Assay intercepts, routes, decides and returns the same response; nothing changes on the agent's side. For the demo everything runs inside Databricks.

Pitch line: *Databricks collects the evidence and enforces the rules. Assay decides what the rules should be.*

3. How we got here

Before the event (ideas only; no code carried over)

- Started as an **agent cost control plane**, then "**certify cheaper models**" (the original Assay).
- Overnight **pre-event lab**: 518 real Claude runs (514 succeeded), \$33.01 of API-equivalent usage, \$0 paid (subscription). Findings, labelled pre-event wherever we quote them: switching one conversation turn from Opus to Sonnet cost 4.1× staying on Opus (the prompt cache is lost); subagents started in parallel each pay the full cold-cache cost; a rarely used setup cost 2.25× per task; cost per task vs Opus: Opus-low –19%, Sonnet –38%, Haiku –70%.

- **The problem with the lab:** every model scored 100% on its clean, synthetic questions, so it could not tell models apart. We needed real, messy work with real ground truth.

Saturday, hour by hour (times from our notes and the commit log, CDT)

Time	What happened
10:00–11:00	Opening ceremony. We learn Databricks already routes between models automatically, but doesn't prove quality.
11:00–13:00	Direction settled ("Path B"): keep the proof engine; give it a real agent. Apache Jira triage chosen because maintainers' own links are free ground truth.
12:45	Team repo created (Ananda-001), README updated (Soham Vazirani).
13:45–14:55	Built in a local folder: Jira ingest, time-honest retrieval, the LLM judge, the statistics engine, a first review app. First real test: 60 tickets × Claude Haiku and Sonnet.
14:30	Brief v1 (docs/thinking/01-brief-1430.md).
15:20	Brief v2: a teammate's Databricks research (what Free Edition offers, what Assay adds on top).
15:40	Brief v3: fact-check of v2. Found the base-rate trap and that we have only 142 duplicate tickets.
15:42	Code moved into the team repo in 9 reviewed commits.
17:09	Mohit: local review workbench (SQLite, one link per approval, restart-safe), frozen evaluation plans, prompt versions.
17:46	Mohit: permission receipts, the learning gate on tasks, the honest-stream sampler, frozen plans.
17:50–18:00	Databricks connected: data into Unity Catalog, review store as a Delta table, app deployed.
18:07	Review notes fixed (reasoning-token cap, one label per action, maintainer pre-fill, "needs N more").
18:05–18:25	Stage 2/3 runs on Databricks Llama 70B / 8B (270 + 90 calls, \$0).
18:30–18:55	AI labelling (gpt-oss-120b + Claude on weak items), two rounds of human spot checks, verdicts.
19:02	App redeployed showing the receipts and verdicts.
19:15–20:10	Dashboard lab (4 designs), then the manager mock (inbox + savings + guided tour). Product reframed as an "AI agent manager".
20:36	Live model switching on Databricks Free Edition.
20:50	Reuse of past reviewer decisions, with proof.

Why these were not pivots

The engine never changed: prove before you trust. What changed was the evidence it runs on (synthetic → real Jira), the platform (laptop → Databricks), and the words (proof engine → agent manager). Every earlier piece still runs: the permission bands are question 1, model certification became live switching, the learning gate is question 3, and reviewer clicks became the precedent memory.

4. The demo agent and its data

Task. For each new Jira ticket, compare it with earlier tickets and say whether it is a **duplicate**, **part of** an umbrella ticket, **related**, or **none**. Acting means writing the link to our own table (we never write to Apache's Jira; in production this step would call the Jira API).

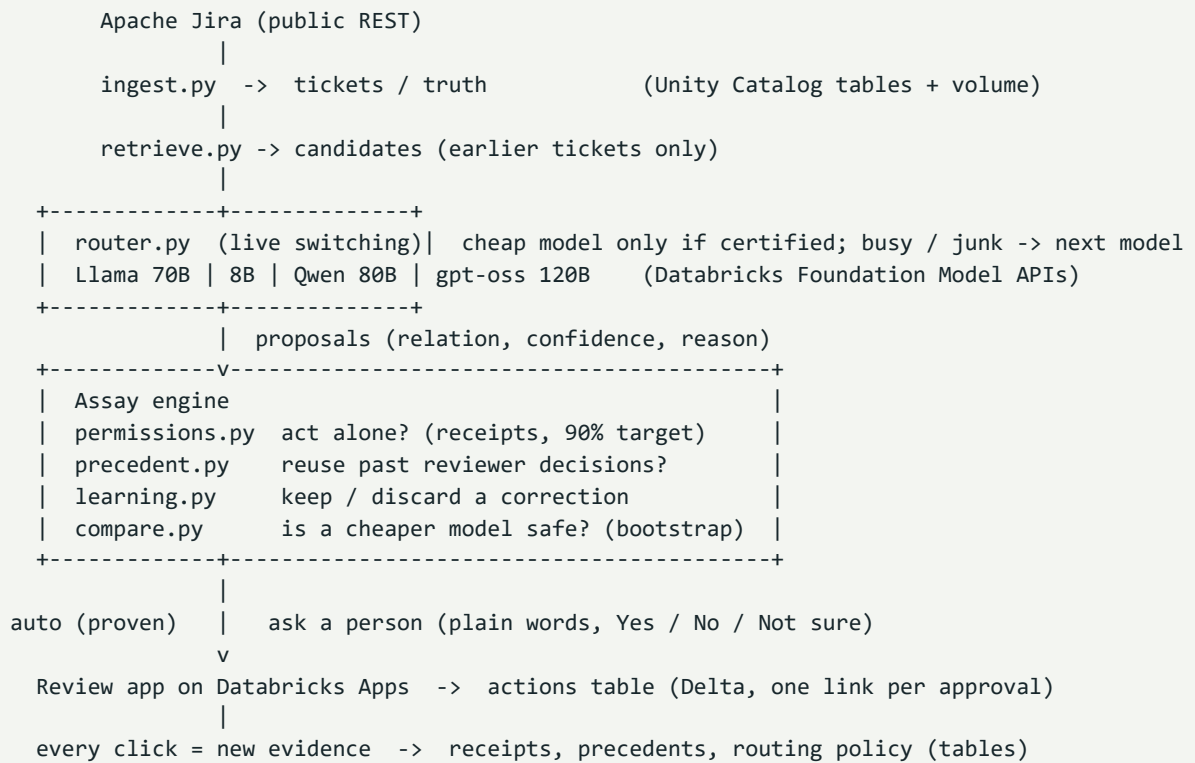
Data (`assay_triage/ingest.py` , public Apache Jira REST, no login):

Fact	Value
Tickets since 2023 (SPARK, FLINK, KAFKA, HIVE)	37,853
Maintainer links used as truth	13,687 (part_of 11,726 · related 1,500 · duplicate 461)
Tickets created from 2025 on (the evaluation window)	16,919 (SPARK 9,073 · FLINK 3,835 · KAFKA 2,763 · HIVE 1,248)
Duplicate tickets in the 2025+ window	142 (≈0.8% of the stream)
Tickets closed as duplicates that never link to the original	48%

Retrieval (`assay_triage/retrieve.py`): TF-IDF over title ×2, components and description; it only ever looks at tickets created *before* the one being judged. A "sibling vote" adds the parents of similar earlier tickets as umbrella candidates. Recall@10 on 2025+ tickets: **duplicate 72.5%**, **part_of 57.2%** (13.6% before the sibling vote), **related 49.4%**. Caveat: a sibling's parent is read from today's data, which is slightly optimistic.

Judge (`assay_triage/judge.py`): one model call judges all 5 shortlisted candidates of a ticket and returns JSON (relation, confidence, reason). Backends: Claude CLI, Anthropic, OpenAI, Databricks. Prompt versions: `v1` , `v2` (siblings are not the umbrella; "related" only for concrete technical links), `bad` (a deliberately poisoned correction for the stress test). Model calls are off unless `ASSAY_ALLOW_MODEL_CALLS=1` .

5. Architecture



Where things run today: models, data tables, the review store and the app run on Databricks. The evaluation and routing scripts run on a laptop and call Databricks. Moving the agent loop into a Databricks notebook or job is on the next-steps list.

6. Databricks integration (Free Edition)

Piece	State
Workspace	Free Edition, one SQL warehouse (Serverless Starter, 2X-Small)
Unity Catalog schema workspace.assay_triage	tickets 37,853 · truth 13,687 · candidates 14,910 · judgments 656 · actions (review decisions) · routing_log 36 · precedents 13
Volume /Volumes/workspace/assay_triage/data	the four JSONL inputs (tickets file is 42 MB, above the 10 MB app-file limit, so the app reads it from here)
Models (Foundation Model APIs)	chat: Llama 3.3 70B, Llama 3.1 8B, Llama 4 Maverick, Qwen3-Next 80B, Qwen3.5 122B, gpt-oss 120B / 20B, Gemma 3 12B; embeddings: GTE, BGE, Qwen3 0.6B. No Claude models on Free Edition.
Databricks App assay	Streamlit workbench; reviews and links written to the Delta actions table by one atomic MERGE per decision (live test: 6 simultaneous accepts → exactly 1 link). Redeploy: scripts/databricks_deploy.py.
AI Gateway	usage tracking is on for our endpoints, but on Free Edition the usage system tables are managed by Databricks and can't be read ("can only be enabled by Databricks"). We record token counts from every response instead. Fallback routing exists only for external models, so live switching is done in our code.
Limits we hit	per-workspace request rate (HTTP 429) on Llama 70B when 5–16 requests run at once; apps stop 24 h after a deploy; outbound internet limited until LinkedIn verification.

Scripts: scripts/databricks_setup.py (schema, volume, tables), scripts/databricks_deploy.py (app), assay_triage/dbx.py (client, SQL through the warehouse, uploads), assay_engine/delta_store.py (review store).

7. Methods, in plain words

Precision with an error bar. "Right k of n times" becomes a one-sided 95% lower bound (exact Clopper-Pearson). An action may go on autopilot only if that lower bound is at least the target, 90% by default.

To prove at least...	with 0 mistakes	1 mistake	3 mistakes
90% (one test)	29 of 29	46	76
90% (engine default: two start points, error split)	36 of 36	54	85
95% (one test)	59 of 59	93	153

No lucky thresholds. Confidence cutoffs are tested from the most confident down, stopping at the first failure (fixed-sequence), so scanning many cutoffs can't produce a lucky pass. For the permission receipts the cutoff is fixed in advance at 0.95, and the error rate is split across the three relation types (Bonferroni).

One decision per ticket. A ticket's action is its most confident non-"none" call; siblings under one umbrella count once. Five candidates of one ticket are not five independent pieces of evidence.

Time-honest. Evaluation uses tickets from 2025 on, and retrieval only sees earlier tickets.

The base-rate trap (and the fix). A sample that is one-third duplicates makes "duplicate" calls look far more precise than on the real stream (about 1% duplicates). So the stream is defined before judging: tickets whose best search match scores at least 0.4575 (the 75th percentile, chosen without looking at outcomes), 4,231 tickets. Plans are drawn uniformly from it and frozen (hash-checked) before any model sees them: permission 90 tickets (seed 11), gate 60 (seed 23), and a version-update stress slice of 30 (seed 37).

No peeking. Prompt changes are evaluated on tickets that were not used to write them.

Learning gate. Before vs after on the same frozen tickets; only tickets whose action changed matter (same action = tie). Exact sign test on fixes vs breaks: KEEP, DISCARD or UNPROVEN.

Label only what matters. Permission receipts use only actions at the 0.95 cutoff; the gate uses only tickets where the two versions disagree. That cut the labelling from about 360 decisions to 107 worksheet rows (2 settled by maintainer links, 105 left).

Precedent reuse. Past decisions grouped by kind of case (read from the two tickets only); reuse switches on when reviewer agreement clears the same 90% lower-bound bar; the reuse is proven by leave-one-out.

8. Experiments and results

8.1 First real test: 60 tickets × Claude Haiku / Sonnet (diagnostic only)

results/2026-09-26-hardness-60/ · command `scripts/judge_eval.py --models haiku,sonnet --n 60 --k 5` (Claude CLI).

Model	Pair accuracy	Duplicate P / R	Part-of P / R	Related precision
Haiku 4.5	53.4%	66.7 / 66.7	57.9 / 68.8	8.4%
Sonnet 5	55.8%	61.1 / 73.3	50.0 / 75.0	7.0%

Model comparison said **REJECT** Haiku: +90.5% cost per task, quality −2.4 points [−5.8, +0.9]. The cost came from about 1,700 hidden "thinking" tokens per Haiku call inside Claude Code (Sonnet used none). That is a finding about our setup, not about Haiku. **Diagnostic only:** the sample was balanced 15/15/15/15, and the true answer was inserted into the shortlist when search missed it (a teammate caught this in review).

8.2 The honest runs on Databricks

results/stage-2-3-runs/README.md · runner scripts/run_stage23_databricks.sh.

Run	Model	Prompt	Tasks completed
permission	Llama 3.3 70B	v2	90 / 90
permission (cheap)	Llama 3.1 8B	v2	80 / 90: 10 unusable JSON answers (11%), kept as failures, not retried
gate	Llama 3.3 70B	v1 and v2	60 / 60 each
stress	Llama 3.3 70B	v1 and bad	30 / 30 each

The 70B's only failures were rate limits, filled by a second pass with back-off. Average tokens per call: 70B 703 in / 167 out; 8B 699 in / 224 out. Cost: \$0 (Free Edition).

Behaviour before any labels: the 70B proposed a link on 89 of 90 tickets, 60 of them "related" (over-linking), and only 2 actions reached the 0.95 cutoff. The 8B called "duplicate" on 48 of 80 and claimed 95%+ confidence on 30 actions. Prompt v2 changed the 70B's action on 14 of 60 tickets. The poisoned correction changed 23 of 30, raising "duplicate" calls from 4 to 25.

8.3 Labels, and how we checked the labellers

- `gpt-oss-120b` on Databricks labelled the worksheet rows blind (never seeing the proposing model, the prompt or its confidence): 104 of 105 done, 1 call failed.
- Claude re-judged the 10 items where gpt-oss was unsure or under 80% confident.
- **Human spot check** on 20 random AI-labelled items, blind, one reviewer (krish):

	Agreement with the AI labels
Round 1 (no definitions on screen)	11 / 20 (55%)
Round 2 (written definitions shown)	11 / 20 (55%, 90% interval 35–74%)
Round 2, duplicate	8 / 9 (89%, interval 57–99%)
Round 2, related	2 / 7
Round 2, part of	0 / 2

Definitions were grounded in maintainer practice: of 6,743 pairs of tickets that bump the same library, 36 are linked, and all 22 duplicate links share the same target version. **Conclusion:** duplicate labels are usable; "related" and "part of" labels are not, so no claim rests on them.

8.4 Verdicts

results/stage-2-3-runs/VERDICTS.md · receipts in results/stage-2-3-runs/receipts-ai/.

Question	Verdict	Evidence	Survives label error?
May Llama 8B auto-link duplicates when it says $\geq 95\%$?	QUIET: no	4 of 19 right (21%); lower bound 5% vs 90%	Yes: even generous flips leave it near 32%
May Llama 70B act alone?	SUGGEST	2 actions at the cutoff (1 right); part-of needs ~ 38 more	n/a: too little evidence, stated as such
Keep prompt v2?	UNPROVEN	fixes 3, breaks 3; same action on 46 of 60	yes: noisier labels can't prove it
Keep the poisoned rule?	DISCARD	fixes 0, breaks 16 (all duplicate calls), $p = 1.5 \times 10^{-5}$	yes: 14 vs 2 would still give $p \approx 0.002$

8.5 Live model switching (assay_engine/router.py, scripts/live_route.py)

Policy (results/routing-policy.json): primary Llama 70B; Llama 8B **not certified** ("right 4 of 19 when it said 95%+ sure; 11% unusable answers"; basis: permission receipt and format failures, not a paired cost comparison); backups Qwen 80B then gpt-oss 120B; answers from uncertified models are held for review.

Burst	Result
12 fresh stream tickets, 6 at a time	12 answered by Llama 70B, no switches
24 fresh stream tickets, 16 at a time	23 by Llama 70B; 1 real live switch: SPARK-51070, Llama 70B busy → Qwen 80B, held for review

All 36 decisions are in results/routing-log.jsonl and the Delta table routing_log. Nothing was simulated: switches only happen on real rate limits or real unusable answers.

8.6 Reusing past reviewer decisions (assay_engine/precedent.py, scripts/precedents.py)

97 past decisions (77 audited AI labels, 18 human, 2 maintainer links), 13 kinds of case.

Kind of case (proposed relation)	Reviewers	Agreement	Status at 90%
duplicate: version upgrade vs a different-version upgrade of the same library	reject	21 / 21 (lower bound 0.87)	ask a person; ≈8 more consistent reviews to prove
duplicate: no recognised pattern	reject	11 / 14	ask a person
related: no recognised pattern	accept	9 / 13	ask a person
related: same-library upgrades	accept	8 / 12	ask a person
related: two sub-tasks of the same parent	accept	8 / 9	ask a person
part of: two sub-tasks of the same parent	reject	5 / 7	ask a person
duplicate: two tickets with the same title	split	2 / 4	ask a person (even identical titles are not safe)

At the 90% bar nothing is automatic yet (leave-one-out: 0 of 97 auto-resolved). At an 85% bar, reported separately and never swapped in silently, the version-upgrade precedent switches on: leave-one-out **21 of 21 right** (lower bound 0.87), and 5 of 175 unreviewed proposals would be auto-rejected. **Demo moment:** a manager reviews about 8 more of those cases in the app, and the precedent switches on in front of the judges. That is the "improves when a human corrects it" loop the Xorbix challenge asks for.

8.7 What the manager would see (numbers used in the mock)

180 tickets read · 89 suggestions · 0 done alone · 360 AI calls at \$0 · 105 checks instead of 360 (–71%) · ≈6.4 hours saved (estimate: 255 checks × 1.5 min) · 31 wrong changes prevented (15 unearned merges + 16 from the blocked rule).

9. Break Card material (The Art of the Break)

Failure	How often	Evidence	What we learned
Over-confidence: "95% sure" duplicates	15 of 19 wrong	8B permission receipt	Never gate autonomy on a model's own confidence
Poisoned correction	broke 16, fixed 0	gate-v1-to-bad receipt	Test every correction before switching it on
Unusable output from the cheap model	10 of 90 tasks (11%)	permission-v2- cheap.jsonl.failures.jsonl	Count format failures as failures when comparing cost
Over-linking by the big model	60 of 90 tickets "related"	permission-v2 run	"Related" must stay suggest-only
Siblings mistaken for the umbrella	part-of overconfidence in the first test	hardness test	Prompt v2 targets it; UNPROVEN so far
The "cheap" model was the expensive one	+90.5% cost	hardness test (Claude CLI)	Control how a model runs before comparing costs
Labellers disagree	55% human-AI agreement overall	spot checks	Audit the labels; claim only where they agree (duplicates 8/9)
Rate limits under load	1 real switch in a 16-way burst	routing log	Live switching must be safe: uncertified backups are held for review
Sample design	balanced classes + inserted answers	code review by a teammate	An enriched sample fakes precision; define the stream first

10. What is built (repo map)

```
assay_triage/  ingest.py (Jira -> tickets, truth) · retrieve.py (time-honest shortlist) · judge.py
(LLM judge,    4 backends, prompt versions, fail-fast retries) · plans.py (frozen, hash-checked
plans) ·
               identity.py (config ids) · dbx.py (Databricks client, SQL, uploads)
assay_engine/  bounds.py · bands.py (precision bands, auto threshold, extra_needed) · compare.py
(paired bootstrap)
               · gate.py · report.py · policy.py (one action per ticket, label keys, maintainer pre-
fill)
               · permissions.py (receipts, can_act, needs_more) · learning.py (task-level gate, ties)
               · local_store.py (SQLite) · delta_store.py (Delta) · router.py (live switching) ·
precedent.py
app/           workbench.py (review workbench, local or Databricks) · label_app.py (blind labelling
page)
               · app.py (first dashboard, superseded)
scripts/      judge_eval.py · prepare_eval.py · prepare_stream.py · run_frozen_eval.py ·
run_stage23_databricks.sh
               · label_actions.py · ai_label.py · quick_label.py (one-key terminal labelling) ·
spot_check.py
               · build_permissions.py · evaluate_correction.py · live_route.py · precedents.py
               · databricks_setup.py · databricks_deploy.py
results/      hardness test · Stage 2/3 plans, runs, labels, receipts, VERDICTS.md · routing policy
+ log · precedents
docs/         SCHEMA.md · DATABRICKS_SETUP.md · STAGE_ONE.md · STAGE_TWO_THREE.md · thinking/
(briefs v1-v3) · this report
tests/        61 tests, all passing
```

About 5,600 lines of Python, all written at the event.

11. How to run it

```
pip install -r requirements.txt
python -m pytest -q                                # 61 passed
# data (public Jira; cached)
python -m assay_triage.ingest && python -m assay_triage.retrieve --all
# Databricks (needs .env with DATABRICKS_HOST, DATABRICKS_TOKEN, DATABRICKS_WAREHOUSE_ID)
python scripts/databricks_setup.py                  # schema, volume, tables
python scripts/databricks_deploy.py                 # the app (re-run within 24 h of judging)
# the honest runs (Free Edition, $0)
bash scripts/run_stage23_databricks.sh
# labels, verdicts
python scripts/label_actions.py ... ; python scripts/ai_label.py <worksheet>
python scripts/quick_label.py results/stage-2-3-runs/spot-check-round2.jsonl --name <you>
python scripts/build_permissions.py ... ; python scripts/evaluate_correction.py ...
# live switching and precedents
ASSAY_ALLOW_MODEL_CALLS=1 python scripts/live_route.py --n 24 --workers 16 --delta
python scripts/precedents.py --clicks --delta
```

12. Team and process

- **People in the commit history:** Ananda-001 (created the repo), Soham Vazirani (README), guruvayoorra (krish), mohithnikesh1 (Mohit). Claude (Opus 5.5) wrote much of the code with the team; those commits carry a Co-Authored-By line.
 - **Event rules we follow:** all code written at the event (ideas from before the event are credited, code is not copied); commits early and often, with real timestamps; no backdating.
 - **Our own rules:** keys only in a git-ignored `.env`; nothing paid runs without a person starting it; every number in the video or Devpost comes from a file in `results/`; say what we couldn't prove.
 - **Review culture:** a teammate's review caught the inserted-answer and pair-counting problems, a statistics bug in the cost comparison (it could certify with a zero-width interval), the reasoning-token cap, and the missing "needs N more". All but the comparison bug are fixed; the comparison verdict is not used anywhere.
-

13. Honest limits: what we do not claim

- No model has earned the right to act alone (no AUTO receipt).
 - Model cost/quality certification is not claimed; the paired comparison has a known bug with identical outcomes and has not been re-run on the Databricks models.
 - "Related" and "part of" labels are unreliable (human–AI agreement 2/7 and 0/2); only duplicate claims are made.
 - Most labels come from an AI labeller audited by one human on 20 items; more human review would tighten everything.
 - One live switch was observed; switching under load depends on Free Edition's rate limits.
 - Precedent reuse is not yet proven at 90%; the 85% figure is reported separately.
 - The agent loop runs on a laptop calling Databricks, not yet as a Databricks job.
 - Four Apache projects only; a sibling's parent is read from today's data (slightly optimistic retrieval).
 - The dashboard for managers exists as a mock; the live app still has the earlier, technical design.
-

14. What's next (in order)

1. **UI:** turn the manager mock into the Databricks app (inbox, savings, trust meters, held-back items, model decisions, the guided tour), reading the tables live; teammates' GitHub repos to be integrated here.
2. **Sync results into tables:** runs, labels, receipts, verdicts and routing policy as Unity Catalog tables.
3. **Run the agent inside Databricks:** a notebook or job reading new tickets from the table and writing decisions back.
4. **The precedent demo:** review the remaining version-upgrade cases in the app and show the precedent switch on.
5. **Story:** Break Card (section 9), 2-minute video, README, Devpost answers, the three mentor visits; confirm the challenge declaration; rotate the API key pasted in chat earlier.

Appendix A. Glossary for non-technical readers

- **Agent:** an AI system that does a job (here: tidying Jira tickets) rather than just chatting.
- **Confidence:** how sure the model *says* it is. Not the same as how often it is right.
- **Precision:** of the times it acted, how often it was right.
- **Lower bound / error bar:** the lowest precision the evidence still supports; we act on this, not the average.
- **Receipt:** a stored, tamper-evident record of a verdict and the evidence behind it.
- **Precedent:** what reviewers decided before on the same kind of case.
- **Stream:** the tickets the agent actually sees, defined before any judging.
- **Databricks Free Edition:** the free version of the data platform we run on (models, tables, app).
- **Unity Catalog / Delta table:** Databricks' governed tables; where our data and decisions live.

Appendix B. Key numbers (cheat sheet)

Real tickets	37,853
Duplicate closures never linked to the original	48%
Model calls on Databricks	360 evaluation + 105 labelling + 36 routed
Cost of those calls	\$0 (Free Edition)
Cheap model "95% sure" duplicates that were right	4 of 19 (21%)
Poisoned correction	broke 16, fixed 0 → DISCARD ($p = 1.5 \times 10^{-5}$)
Prompt v2	fixed 3, broke 3 → UNPROVEN
Human–AI label agreement on duplicates	8 of 9
Checks needed with Assay vs checking everything	105 vs 360 (–71%)
Live switches observed	1 (Llama 70B busy → Qwen 80B, held for review)
Strongest precedent	21 of 21 rejections; ≈8 reviews from proven at 90%
Tests	61 passing